

D. Merge, Rebase, Squash!

Problem ID: Git

Time Limit: 2.5s

Memory Limit: 512MiB



Force push 1



Force push 2

Figure 1: 不要學 DB force push (他現在已經不會了)

tobiichi3227 是 TOJ 的維護者，TOJ 整個程式碼由 H 個獨立的 **hunk** 組成，每個 hunk 的狀態一開始都是 0，在 v2.0 分支上面。

有天王德宏想給 TOJ 貢獻一個酷炫的功能，於是王德宏很高興的開啟了一個新的分支 feat/fancy 並在上面建立了 N 個 commit。

王德宏想讓功能進到 TOJ 的 v2.0，於是請 tobiichi3227 來 review，tobiichi3227 覺得 N 個 commit 太多了，應該用 squash 把 commit 減少到只有 K 個。

當 squash 完成後，會將 feat/fancy rebase 到 v2.0，王德宏發現會有 conflict，所以他想要最小化解決 conflict 的成本，但王德宏要去比國際地理奧林匹亞 (IGGO, 全名 International GeoGuesser Olympiad) 沒空了，於是交給在看題目的你。

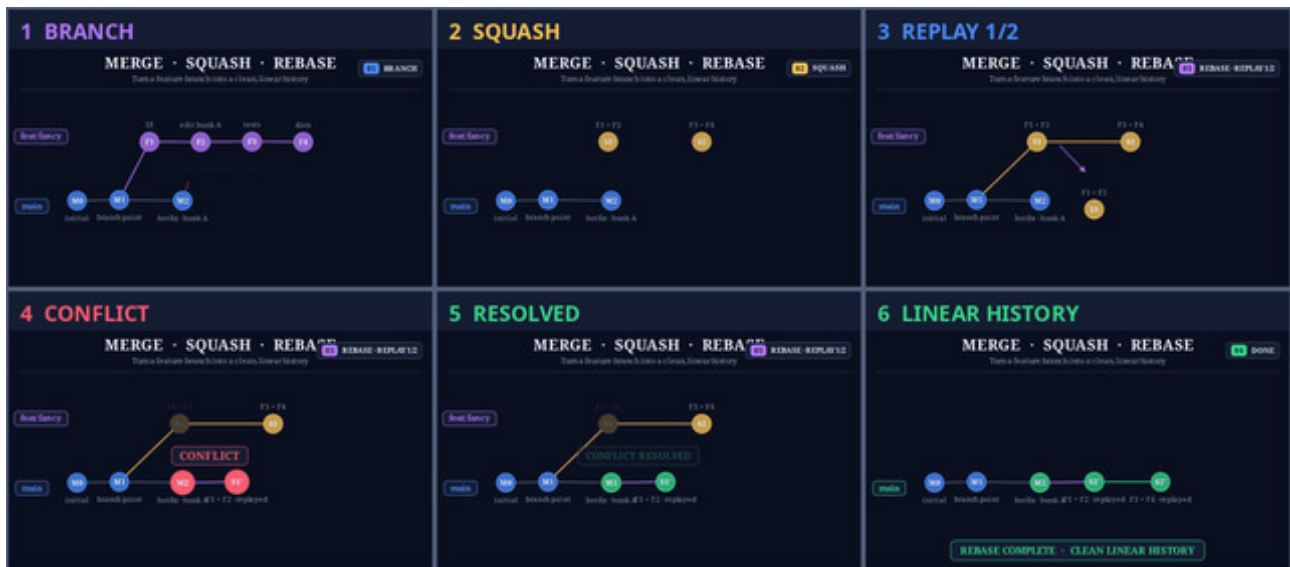


Figure 2: Branch、squash、rebase 與 conflict resolution 的執行過程

若使用支援 PDF 附件的 Firefox，可[點我下載完整動畫](#)；Chrome 與 Edge 請直接閱讀上方分鏡。

— Branch 狀態 —

v2.0 branch 上第 h 個 hunk 的最終內容為 U_h 。

feat/fancy branch 上共有 N 個 commits。第 i 個 commit 會將第 x_i 個 hunk 的內容改成 v_i 。

令 $S_i[h]$ 表示執行完 feat/fancy branch 的前 i 個 commits 後，第 h 個 hunk 的內容。

初始時： $S_0[h] = 0$

第 i 個 commit 執行後： $S_i[x_i] = v_i$

其他 hunk 的內容不變。

– Squash –

原本的 N 個 commits 依照順序排列：

commit 1, commit 2, ..., commit N

你需要在這些 commits 之間放置 $K - 1$ 個切點，將它們分成恰好 K 組。

每一組中的所有 commits 會被合併成一個新的 **squash commit**。

例如，假設共有 7 個 commits，且要分成 3 組，可以選擇：

```
[commit 1, commit 2]
[commit 3, commit 4, commit 5]
[commit 6, commit 7]
```

也可以簡寫成：

```
[1, 2] [3, 4, 5] [6, 7]
```

這代表：

- 第一個 squash commit 包含原本的 commits 1 到 2
- 第二個 squash commit 包含原本的 commits 3 到 5
- 第三個 squash commit 包含原本的 commits 6 到 7

為了描述一般情況，令： $0 = p_0 < p_1 < p_2 < \dots < p_K = N$

其中 p_g 表示第 g 組的最後一個 commit 編號。

因此，第 g 個 squash commit 包含： $p_{g-1} + 1, p_{g-1} + 2, \dots, p_g$

例如前面的分組：

```
[1, 2] [3, 4, 5] [6, 7]
```

對應： $p_0 = 0, p_1 = 2, p_2 = 5, p_3 = 7$

考慮第 g 個 squash commit，以及某個 hunk h 。

在這組 commits 開始以前，hunk h 的內容是： $S_{p_{g-1}}[h]$

執行完這組中的所有 commits 後，hunk h 的內容是： $S_{p_g}[h]$

因此，這個 squash commit 對 hunk h 的效果為： $S_{p_{g-1}}[h] \rightarrow S_{p_g}[h]$

也就是只保留這一組 commits 執行前與執行後的差異，中間經過的狀態不會出現在 squash commit 中。

例如，某個 hunk 在一組 commits 中依序發生：

1 -> 3 -> 5 -> 2

那麼 squash 後只會留下：

1 -> 2

中間的 3 與 5 都不會出現在 squash commit 中。

若一組 commits 執行前後，某個 hunk 的內容相同，即： $S_{p_{g-1}}[h] = S_{p_g}[h]$

則這個 squash commit 不會修改該 hunk。

– Rebase 與 Conflict –

完成 squash 後，tobiichi3227 會依照原本的順序，將這 K 個 squash commits 逐一套用到 v2.0 branch 上。

考慮其中一個 squash commit 對 hunk h 的修改： $\text{old} \rightarrow \text{new}$

其中：- old：這個 squash commit 預期修改前的內容；- new：這個 squash commit 想要修改成的內容；- current：目前 v2.0 branch 上實際存在的內容。

例如： $\text{old} = 3$, $\text{new} = 7$, $\text{current} = 5$

代表這個 squash commit 原本是在內容為 3 的基礎上，將它改成 7，但目前 v2.0 branch 上的內容實際是 5。

沒有修改

若： $\text{old} = \text{new}$

代表這個 squash commit 執行前後，該 hunk 的內容沒有改變。

因此不需要進行任何操作，current 也不會改變。

情況一：目前內容正是預期的舊內容

若： $\text{current} = \text{old}$

代表目前內容與這個 patch 預期的起點相同，因此可以正常套用修改。

套用後： $\text{current} \leftarrow \text{new}$

例如：

```
patch:    3 -> 7
current: 3
```

可以正常套用，完成後 current 變成 7。

情況二：目前內容已經是目標內容

若： $\text{current} = \text{new}$

代表 v2.0 branch 上已經存在這個 squash commit 想要產生的結果。

因此不會發生 conflict，也不需要再次修改。

例如：

```
patch:    3 -> 7
current: 7
```

雖然目前內容不是 patch 預期的舊內容 3，但它已經等於目標內容 7，所以可以直接視為這項修改已經存在。

current 維持為 7。

情況三：目前內容既不是舊內容，也不是新內容

若： $\text{current} \neq \text{old}$

且： $\text{current} \neq \text{new}$

代表 v2.0 branch 上的內容與這個 squash commit 預期的內容不同，而且也不是它想要修改成的結果。

此時會發生 conflict，例如：

patch: 3 -> 7

current: 5

目前內容 5 既不是舊內容 3，也不是新內容 7，因此發生 conflict。

解決 hunk h 的 conflict 需要支付 w_h 的成本。

解決 conflict 後，採用 squash commit 的修改結果： $\text{current} \leftarrow \text{new}$

在前面的例子中，解決後 current 會變成 7。

同一個 squash commit 可能同時修改多個 hunks。每個 hunk 是否發生 conflict 都要分別判斷，發生 conflict 的成本也要分別計算。

— 輸入 —

第一行包含三個整數 N, H, K ，分別表示：

- feat/fancy branch 上的 commit 數量；
- hunk 的數量；
- squash 後必須保留的 commit 數量。

第二行包含 H 個整數 U_h ($1 \leq h \leq H$)

其中 U_h 表示 v2.0 branch 上第 h 個 hunk 的最終內容。

第三行包含 H 個整數 w_h ($1 \leq h \leq H$)

其中 w_h 表示解決第 h 個 hunk 的 conflict 所需的成本。

接下來 N 行，第 i ($1 \leq i \leq N$) 行包含兩個整數 x_i 與 v_i
表示第 i 個 commit 將第 x_i 個 hunk 的內容改成 v_i 。

— 輸出 —

輸出一個整數，表示最小的 conflict 解決總成本。

— 輸入限制 —

- $1 \leq N, H \leq 200000$
- $1 \leq K \leq \min(N, 30)$
- $1 \leq U_h \leq 10^9$
- $1 \leq w_h \leq 10^9$
- $1 \leq x_i \leq H$
- $1 \leq v_i \leq 10^9$
- 對於每個 commit $i : v_i \neq S_{i-1}[x_i]$
- 保證每個 commit 都會實際改變該 hunk 的內容，且 feat/fancy branch 不會將任何 hunk 改回 0。

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	8	$N, H \leq 15$
3	8	$K = 1$
4	8	$K = N$
5	10	每個 hunk 最多只被修改一次
6	26	$N \leq 1000$
7	40	無限制

— 範例輸入 —

```
5 2 2
1 4
5 3
1 1
2 2
1 2
2 4
1 3
```

— 範例輸出 —

```
0
```

— 範例說明 —

feat/fancy branch 上兩個 hunk 的狀態依序如下：

```
commit 0: (0, 0)
commit 1: (1, 0)
commit 2: (1, 2)
commit 3: (2, 2)
commit 4: (2, 4)
commit 5: (3, 4)
```

可以將 commits 分成：

[1] [2, 3, 4, 5]

第一個 squash commit 將 hunk 1：

0 -> 1

套用到 v2.0 branch 時，hunk 1 的目前內容已經是 1，因此不會發生 conflict。

第二個 squash commit 的效果為：

hunk 1 : 1 -> 3

hunk 2 : 0 -> 4

此時 hunk 1 的目前內容為 1，可以正常套用。

hunk 2 的目前內容已經是 4，因此也不會發生 conflict。

總成本為 0。

— 範例輸入 —

```
2 1 1
1
7
1 1
1 2
```

— 範例輸出 —

```
7
```

— 範例說明 —

因為 $K = 1$ ，兩個 commits 必須全部 squash 在一起。
Squash 後的效果為：

```
0 -> 2
```

但 v2.0 branch 上該 hunk 的內容為 1。

目前內容既不是 patch 的舊內容 0，也不是新內容 2，因此發生 conflict，成本為 7。

— 範例輸入 —

```
2 1 2
1
7
1 1
1 2
```

— 範例輸出 —

```
0
```

— 範例說明 —

因為 $K = 2$ ，兩個 commits 分別套用。
第一個 commit 的效果為：

```
0 -> 1
```

而 v2.0 branch 上目前的內容已經是 1，因此不會發生 conflict。

第二個 commit 的效果為：

1 -> 2

此時目前內容為 1，可以正常套用。

因此總成本為 0。

— Git 小教室 —

下面會用一個小型 repository 實際重現題目中的 branch、squash、rebase 與 conflict。整份教學都能在**沒有網路**的環境執行：toj.git 是建立在同一台電腦上的本機 remote，不會連線到 GitHub。命令以 Bash 或 Git Bash 為例；程式碼框中的命令不用輸入開頭的提示符號。

Git 在記錄什麼？

- **working tree**：目前資料夾中實際看到的檔案。
- **staging area**：下一個 commit 準備收錄的內容。
- **commit**：repository 在某個時間點的快照，以及指向前一個 commit 的資訊。
- **branch**：指向某個 commit、並會隨新 commit 前進的名稱，例如 v2.0、feat/fancy。
- **HEAD**：目前 checkout 的 branch 或 commit。
- **remote**：另一個 repository 的別名；remote 可以在網路上，也可以只是本機路徑。

最常使用的命令如下：

git init	建立 repository
git status	查看目前 branch、已修改與已 stage 的檔案
git diff	查看尚未 stage 的差異
git diff --staged	查看已 stage、將進入下一個 commit 的差異
git add <file>	將檔案的目前內容放進 staging area
git commit -m "<message>"	建立 commit
git log --graph --oneline --all	用簡圖查看所有 branch 的歷史
git branch	列出 branch
git switch <branch>	切換 branch
git switch -c <branch>	建立並切換到新 branch
git merge <branch>	把指定 branch 合併到目前 branch
git rebase <branch>	把目前 branch 的 commits 逐一重播到指定 branch

`git commit -am "<message>"` 是 `git add` 已追蹤檔案再 `git commit` 的簡寫，但**不會加入新建立、尚未追蹤的檔案**。不確定目前狀態時，先執行 `git status` 通常最安全。

1. 建立完全離線的 repository

先建立一個新的空資料夾。`toj.git` 模擬遠端的 TOJ repository，`work` 則是實際工作的 repository：

```
mkdir git-classroom
cd git-classroom
git init --bare toj.git
git init -b v2.0 work
cd work
```

bare repository 沒有 working tree，適合當作大家交換 commits 的 remote。接著只替這個練習設定作者資料；`example.invalid` 是保留給範例使用的網域，不需要網路：

```
git config user.name "Git Student"
git config user.email "student@example.invalid"
```

用兩個一行文字檔模擬兩個 hunk，並建立共同的起始 commit：

```
printf "0\n" > hunk-1.txt
printf "0\n" > hunk-2.txt
git add hunk-1.txt hunk-2.txt
git status
git commit -m "base: all hunks are 0"
git tag base
```

base tag 只是替這個 commit 取一個固定名稱，之後進行 squash 時便不必抄寫隨機產生的 commit hash。

2. 在 feat/fancy 建立四個 commits

從共同起點建立新 branch：

```
git switch -c feat/fancy
```

依序建立四個 commits。這對應題目中同一個 hunk 可能被多次修改：

```
printf "3\n" > hunk-1.txt
git commit -am "fancy 1: hunk 1, 0 to 3"
```

```
printf "4\n" > hunk-2.txt
git commit -am "fancy 2: hunk 2, 0 to 4"
```

```
printf "5\n" > hunk-1.txt
git commit -am "fancy 3: hunk 1, 3 to 5"
```

```
printf "7\n" > hunk-1.txt
git commit -am "fancy 4: hunk 1, 5 to 7"
```

可以查看目前的四個 feature commits：

```
git log --oneline --decorate
```

將本機 bare repository 命名為 origin，並保存 squash 前的 branch：

```
git remote add origin ../toj.git
git push -u origin feat/fancy
```

-u 會記住本機 feat/fancy 對應到 origin/feat/fancy。這裡的 remote 是 ../toj.git，因此 push 全程只會存取本機檔案。

3. 讓 v2.0 同時向前進

切回 v2.0，令它的兩個 hunk 最終為 (5, 4)：

```
git switch v2.0
printf "5\n" > hunk-1.txt
printf "4\n" > hunk-2.txt
git commit -am "v2.0: set final hunk states"
git push -u origin v2.0
git log --graph --oneline --decorate --all
```

現在兩個 branches 從 base 分岔：

```

          fancy 1 -- fancy 2 -- fancy 3 -- fancy 4 (feat/fancy)
        /
base: (0, 0) --
        \
          v2.0: (5, 4) (v2.0)

```

4. Merge 與 Rebase 有什麼不同？

merge 會把另一條 branch 的最終結果合進目前 branch；若兩邊都有新 commits，通常會另外建立一個有兩個 parents 的 merge commit。先建立不影響正式流程的 merge-demo：

```
git switch -c merge-demo v2.0
git merge feat/fancy
```

兩邊都把 hunk-1.txt 從 0 改成不同內容，因此 Git 會回報 conflict。兩邊都把 hunk-2.txt 改成 4，所以該檔案可以自動合併。用以下命令可以查看未解決的檔案：

```
git status
git diff
```

這次只是在比較 merge，所以放棄它並回到 feature branch：

```
git merge --abort
git switch feat/fancy
```

相對地，rebase v2.0 會找出 feat/fancy 與 v2.0 的共同祖先，將 feature commits 暫時取下，再按原順序逐一套用到新版 v2.0 後面。因此 rebase 後的歷史通常較直線；在這個已經分岔、確實需要 replay 的例子中，commit 的 parent 與內容雜湊改變，commit hash 也會改變。

5. 把四個 commits squash 成兩個

題目要求把連續的 commits 分組。這裡示範 [1, 2] [3, 4]，也就是令 $K = 2$ 。在 feat/fancy 上執行互動式 rebase：

```
git rebase -i base
```

編輯器會依照由舊到新的順序列出四行，commit hash 會因電腦與執行時間而不同：

```
pick <hash 1> fancy 1: hunk 1, 0 to 3
pick <hash 2> fancy 2: hunk 2, 0 to 4
pick <hash 3> fancy 3: hunk 1, 3 to 5
pick <hash 4> fancy 4: hunk 1, 5 to 7
```

把每一組中除了第一個以外的 pick 改成 squash：

```
pick    <hash 1> fancy 1: hunk 1, 0 to 3
squash  <hash 2> fancy 2: hunk 2, 0 to 4
pick    <hash 3> fancy 3: hunk 1, 3 to 5
squash  <hash 4> fancy 4: hunk 1, 5 to 7
```

儲存並關閉編輯器；若 Git 再要求編輯合併後的 commit message，確認內容後同樣儲存並關閉。第一個 squash commit 的效果為：

```
hunk 1: 0 -> 3
hunk 2: 0 -> 4
```

第二個 squash commit 的效果為：

```
hunk 1: 3 -> 7
```

中間的 3 -> 5 -> 7 已經縮成 3 -> 7。確認 base 後面現在恰有兩個 commits：

```
git rev-list --count base..feat/fancy
git log --oneline --reverse base..feat/fancy
```

第一個命令應輸出 2。真實 repository 若沒有事先建立 base tag，可以先用 git merge-base v2.0 feat/fancy 找共同祖先，再將輸出的 commit hash 交給 git rebase -i。

6. Rebase 並解決 conflict

把這兩個 squash commits 逐一 replay 到 v2.0：

```
git rebase v2.0
```

套用第一個 squash commit 時：

- hunk-1 的 patch 是 0 -> 3，但 v2.0 的 current 是 5，因此發生 conflict。
- hunk-2 的 patch 是 0 -> 4，而 v2.0 的 current 已經是 4，因此不發生 conflict。

Git 會停下來等待處理。查看狀態與 conflict markers：

```
git status
cat hunk-1.txt
cat hunk-2.txt
```

hunk-1.txt 會包含以下幾個部分（左欄的說明文字不在實際檔案中）：

```
目前版本的起點標記 | <<<<<<< HEAD
目前版本的內容      | 5
分隔線              | =====
正在 replay 的內容   | 3
feature 版本的標記   | >>>>>>> fancy 1: hunk 1, 0 to 3
```

<<<<<<<、=====、>>>>>>> 是 Git 加入的標記，不是檔案原本的內容。題目規定解決 conflict 後採用 squash commit 的 new，所以刪除標記並把檔案內容改成 3：

```
printf "3\n" > hunk-1.txt
git add hunk-1.txt
git rebase --continue
```

git add 在這裡代表「這個檔案的 conflict 已處理完成」。若 --continue 開啟 commit message 編輯器，直接確認、儲存並關閉即可。接著第二個 squash commit 的 3 -> 7 可以正常套用，rebase 便會完成。

```
git status
cat hunk-1.txt
cat hunk-2.txt
git rev-list --count v2.0..feat/fancy
git log --graph --oneline --decorate v2.0 feat/fancy
```

最後應得到 `hunk-1.txt = 7`、`hunk-2.txt = 4`，而且 `v2.0` 後面恰有兩個 feature commits。若想放棄整次 rebase 並回到開始前，可以在尚未完成時使用：

```
git rebase --abort
```

不要在不了解目前 commit 的情況下使用 `git rebase --skip`；它會直接丟掉正在 replay 的 commit。

7. 為什麼 rebase 後需要 force push？

remote 上仍保存 squash 前的四個 commits，但本機已把它們改寫成另外兩個 commits。普通 push 只允許 fast-forward，因此以下命令應被拒絕：

```
git push origin feat/fancy
```

確認沒有人在 remote 上加入你尚未看過的 commit 後，使用：

```
git push --force-with-lease origin feat/fancy
```

`--force-with-lease` 只會在 remote branch 仍是本機所預期的版本時覆寫它；若別人已經更新 remote，push 會被拒絕。它通常比無條件覆寫的 `--force` 安全，但改寫共享 branch 前仍應先和其他協作者確認。

題目模型與真實 Git 的差異

本題刻意把程式碼簡化成互相獨立的 hunks，並用 `old`、`new`、`current` 三個整數定義是否 conflict。真實 Git 會考慮檔案中的多行 context、相鄰修改、檔案新增刪除與重新命名等資訊，不能在所有情況下只靠三個值完整描述。上面的練習把每個 hunk 放在獨立的一行文字檔中，因此能重現本題所描述的三種結果；解題時仍應以題目正式定義為準。